INTERVIEW READY

20 SQL PATTERNS TO MASTER

Stop writing messy queries. Start crushing interviews.

1. Left Join Exclusion
2. Window Functions
3. Engagement Metrics
4. Ordering & Limit
5. Data Auditing
6. Retention Analysis
7. Compensation Stats
8. Sales Reporting
9. Project Tracking
10. HR Analytics
11. Scalar Analysis
12. Inventory Cleanup
13. Trend Analysis
14. Recruitment Metrics
15. Resource Allocation
16. Null Handling
17. Catalog Management
18. Payroll Audit
19. High Value Targeting
20. Bulk Orders

Query 1

AMAZON

EMPTY DEPARTMENTS

Identify departments that currently have zero employees

Left Join Exclusion

```
-- LEFT JOIN keeps all departments.
-- WHERE e.id IS NULL filters for non-matches.

SELECT d.department_name
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
WHERE e.id IS NULL;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 2

ORACLE

TOP EARNERS BY DEPT

Find the employee with the maximum salary in each department

Window Functions

```sql
-- RANK() assigns a ranking within each partition.
-- Selecting rnk=1 gets the top salary per dept.

SELECT department_id, name, salary
FROM (
  SELECT *, RANK() OVER (
    PARTITION BY department_id
    ORDER BY salary DESC
  ) as rnk
  FROM employees
) sub
WHERE rnk = 1;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 3

ENGAGEMENT

DAILY ACTIVE CUSTOMERS

Customers with orders on EVERY day in the last week

Count Distinct

```sql
-- Filter for last 7 days first.
-- Count distinct dates to ensure daily activity.

SELECT customer_id
FROM orders
WHERE order_date ≥ CURRENT_DATE - INTERVAL '7 days'
GROUP BY customer_id
HAVING COUNT(DISTINCT CAST(order_date AS DATE)) = 7;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 4

Supply Chain

MAXIMUM SINGLE ORDER

Product sold in the highest quantity in a single order

Ordering & Limit

```sql
-- Find max quantity per product.
-- Order descending and take the top 1.

SELECT product_id, MAX(quantity) as max_qty
FROM sales
GROUP BY product_id
ORDER BY max_qty DESC
LIMIT 1;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 5

AUDITING

DATA ANOMALY DETECTION

Employees who joined BEFORE their department was created

Join & Comparison

```
-- Join employees to departments.
-- Compare hire_date vs department creation_date.

SELECT e.name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
WHERE e.hire_date < d.creation_date;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 6

RETENTION

LOYAL MULTI-YEAR CUSTOMERS

Customers with sales in at least 3 different years

Extract & Count

```sql
-- Extract Year from date.
-- Count DISTINCT years to find long-term loyalty.
SELECT customer_id
FROM sales
GROUP BY customer_id
HAVING COUNT(DISTINCT EXTRACT(YEAR FROM sale_date)) ≥ 3;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 7

COMP ANALYSIS

UNIQUE COMP POSITION

Employees with salary > Company Avg BUT < Dept Avg

CTEs & Joins

```
-- Calculate Company Avg and Dept Avg in CTEs.
-- Filter main query against these pre-calculated values.

WITH CompAvg AS (SELECT AVG(salary) as c_avg FROM employees),
DeptAvg AS (SELECT department_id, AVG(salary) as d_avg
            FROM employees GROUP BY 1)
SELECT e.*
FROM employees e
JOIN DeptAvg d ON e.department_id = d.department_id
JOIN CompAvg c ON e.salary > c.c_avg AND e.salary < d.d_avg;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 8

REPORTING

CUSTOMER ANNUAL SPENDING

Average order amount per customer per year

Multi-Column Grouping

```sql
-- Group by both ID and Year.
-- Calculates stats for every customer-year combination.

SELECT customer_id,
  EXTRACT(YEAR FROM order_date) as year,
  AVG(amount)
FROM orders
GROUP BY customer_id, year;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 9

TRACKING

HIGH-VALUE PROJECT TEAM

Employees worked on a project with budget > $1,000,000

Join & Filter

```
-- Chain joins: Emp -> Assignment -> Project.
-- Filter on the final Project table attribute.

SELECT DISTINCT e.name
FROM employees e
JOIN project_assignments pa ON e.id = pa.employee_id
JOIN projects p ON pa.project_id = p.project_id
WHERE p.budget > 1000000;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 10

HR ANALYTICS

LAST PROMOTION DATES

Most recent promotion date per employee

Max Aggregation

```sql
-- Standard Group By pattern.
-- MAX() finds the latest date for each group.

SELECT employee_id, MAX(promotion_date)
FROM promotions
GROUP BY employee_id;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 11

SPENDING

ABOVE-AVERAGE SPENDERS

Customers with total purchases > Average order amount

Agg vs Scalar

```
-- Compare a Group Aggregate (SUM)
-- against a Scalar Subquery result (AVG).

SELECT customer_id, SUM(amount)
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > (SELECT AVG(amount) FROM orders);
```

Anandnarayanan S

Follow to become SQL interview ready

Query 12

INVENTORY

UNSOLD PRODUCTS

Products NEVER ordered (Inventory Cleanup)

NOT IN

```sql
-- Find IDs in Product table
-- that are completely missing from Sales table.

SELECT product_id FROM products
WHERE product_id NOT IN (
  SELECT DISTINCT product_id FROM sales
);
```

Anandnarayanan S

Follow to become SQL interview ready

Query 13

TRENDS

LOWEST SALES MONTH

Month with the lowest sales in the past year

Date Trunc

```sql
-- Truncate dates to Month precision.
-- Sort ASC to find the lowest sum.

SELECT DATE_TRUNC('month', sale_date) as mth,
  SUM(amount) as sales
FROM sales
WHERE sale_date ≥ CURRENT_DATE - INTERVAL '1 year'
GROUP BY mth
ORDER BY sales ASC
LIMIT 1;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 14

RECRUITMENT

MONTHLY HIRING PATTERNS

Number of employees hired each month in the last year

Date Grouping

```
-- Aggregating by Month buckets.
-- Useful for spotting seasonal hiring trends.

SELECT DATE_TRUNC('month', hire_date) as mth,
  COUNT(*)
FROM employees
WHERE hire_date ≥ CURRENT_DATE - INTERVAL '1 year'
GROUP BY mth
ORDER BY mth;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 15

RESOURCES

MOST ACTIVE DEPARTMENT

Department with the highest number of projects

Count & Limit

```
-- Standard 'Top 1' Pattern.
-- Count items, sort descending, limit to 1.

SELECT department_id, COUNT(*) as proj_count
FROM projects
GROUP BY department_id
ORDER BY proj_count DESC
LIMIT 1;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 16

Benefits

Employees without dependents

Employees who do NOT have dependents

Left Join Null

```
-- Left join dependents to employees.
-- NULL dependent_id means no match found.

SELECT e.name
FROM employees e
LEFT JOIN dependents d ON e.id = d.employee_id
WHERE d.dependent_id IS NULL;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 17

CATALOG

CATEGORY PERFORMANCE

Total sales per category (including 0 sales)

Left Join Chain

```
-- COALESCE handles NULL sums for zero sales.
-- Left Joins ensure empty categories appear.

SELECT c.category_name, COALESCE(SUM(s.amount), 0)
FROM categories c
LEFT JOIN products p ON c.category_id = p.category_id
LEFT JOIN sales s ON p.product_id = s.product_id
GROUP BY c.category_name;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 18

AUDIT

PROMOTION ANOMALIES

Employees promoted but salary didn't increase

Join & Comparison

```sql
-- Check latest promotion via Subquery.
-- Flag if current salary is not higher than before.

SELECT e.id, e.name
FROM employees e
JOIN promotion_history p ON e.id = p.employee_id
WHERE e.salary ≤ p.salary_before
AND p.promotion_date = (
  SELECT MAX(promotion_date) FROM promotion_history
  WHERE employee_id = e.id
);
```

Anandnarayanan S

Follow to become SQL interview ready

Query 19

TARGETING

PREMIUM SHOPPERS

Customers with average order amount > $500

Having Clause

```
-- WHERE filters rows. HAVING filters groups.
-- Used here to filter by aggregated AVG.

SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING AVG(amount) > 500;
```

Anandnarayanan S

Follow to become SQL interview ready

Query 20

LOGISTICS

BULK ORDERS

Orders where total quantity exceeds 100 units

Aggregation

```
-- Calculate total items per order ID.
-- Filter for bulk sizes > 100.

SELECT order_id
FROM order_items
GROUP BY order_id
HAVING SUM(quantity) > 100;
```

Anandnarayanan S

Follow to become SQL interview ready

YOU MADE IT!

Hi, I'm Anand! 👋

WAS THIS USEFUL?

Mastering these 20 patterns covers 80% of SQL interview questions.

Repost to your network

YOU'RE NOW INTERVIEW READY!